

Deploying SokoPay to the DigitalOcean droplet (third project, pilot test)

This guide adds SokoPay to the **existing DigitalOcean droplet** as a **third project**, next to the two already running there. The code is pulled straight from the GitHub repository **SokoPay/sokopay-platform**.

SokoPay runs the way it will in production: - **PostgreSQL 16** database. The local demo's SQLite file is only a laptop convenience. - **Redis**, for the cache and task queue. - **Celery** workers and scheduler. - **HTTPS** through the droplet's existing web server.

When testing is finished, section 13 removes SokoPay completely **without touching the other two projects**.

Test data only. SokoPay runs its `staging` profile:

- **Same as production:** HTTPS only, secure cookies, encryption key required, PostgreSQL, JSON logs, debug off.
- **One difference:** it uses the **mock payment partner** until Korba or Nsano is contracted and certified.

Every page shows a "TEST ENVIRONMENT" banner. Don't enter real customers' data or real Ghana Card numbers. The droplet is outside Ghana, which is fine for demo data but not for real customer data without Bank of Ghana and Data Protection Commission approval.

How SokoPay stays separate from the other two projects

Concern	How it's handled
Names	Everything is in the Docker Compose project <code>sokopay</code> : containers, volumes and networks are all prefixed <code>sokopay-</code> .
Database	SokoPay gets its own PostgreSQL container and volume . It doesn't use, or need access to, the other projects' databases.
Ports	PostgreSQL and Redis publish no ports . The web app listens only on 127.0.0.1:8100 (configurable). Nothing new appears on the internet.
Web traffic	The droplet's existing Nginx or Caddy gets one more site (a subdomain) that forwards to 127.0.0.1:8100. The other sites aren't changed.
Memory	Every SokoPay container has a memory limit (about 3 GB in total), so it can't starve the other projects.
Removal	<code>wipe.sh</code> deletes only SokoPay's containers, volumes, image and files. There is no global <code>docker system prune</code> , and the droplet is not destroyed.

Never run `docker system prune` , `docker volume prune` or `docker compose down` from another folder on this droplet. They can delete the other projects' data. Always run SokoPay commands from `/opt/sokopay/deploy` .

1. What you need

Item	Notes
SSH access to the droplet	The user you normally log in with (this guide calls it <code>deploy</code>). It needs <code>sudo</code> and permission to run <code>docker</code> .
The SokoPay domain	<code>sokopay.theagbeko.com</code> . Its A record already points at <code>137.184.192.212</code> (section 4). The phone apps need real HTTPS.
GitHub	The repository <code>SokoPay/sokopay-platform</code> is public for now, so no access setup is needed (section 5).
Free resources	About 3 GB of RAM and 10 GB of disk for SokoPay (section 2).

2. Check the droplet first

Log in, then look at what's already there:

```
ssh deploy@<droplet-ip>

free -h                # memory: SokoPay needs ~3 GB available
df -h /                # disk: SokoPay needs ~10 GB free
nproc                  # CPUs
docker --version && docker compose version # Compose must be v2.20 or newer
docker ps --format 'table {{.Names}}\t{{.Ports}}\t{{.Status}}' # what's running in Docker
sudo ss -ltnp | grep -E ':(80|443|8100) ' # who uses 80, 443, 8100
```

Decide from the results:

You see	Do this
Less than ~3 GB of memory free	Resize the droplet in DigitalOcean (Droplet, then Resize , CPU and RAM only). A short reboot affects all three projects, so pick a quiet time. A 4 vCPU / 8 GB droplet suits three projects.
Port 8100 already used	Pick another free port (for example 8110) and use it as <code>SOKOPAY_WEB_PORT</code> in section 6 and in the proxy config in section 7.
Nginx on 80/443 (in <code>ss : nginx</code>)	Use section 7A .
Caddy on 80/443 (in <code>ss : caddy</code> , on the host)	Use section 7B .
A proxy container on 80/443 (Traefik, nginx-proxy, Caddy in Docker)	Use section 7C .
Nothing on 80/443	Use section 7D (SokoPay's own Caddy).
Docker not installed, or Compose older than v2.20	<code>curl -fsSL https://get.docker.com \ sudo sh</code> . This upgrades Docker in place and keeps existing containers. Then <code>sudo usermod -aG docker deploy</code> and log in again.

Don't change the droplet's SSH settings, time zone or firewall defaults for SokoPay; the other projects rely on them. Ports 80 and 443 are already open, since the other projects use them.

3. Swap (only if the droplet has none)

```
swapon --show          # prints nothing = no swap
# If empty: add 2 GB, which helps the SokoPay image build without squeezing the others
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon
/swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

4. The domain: sokopay.theagbeko.com

SokoPay's address is `sokopay.theagbeko.com`. Its DNS A record already points at **137.184.192.212**. Check that this is the droplet:

```
curl -4 -s ifconfig.me; echo          # on the droplet: should print 137.184.192.212
ping -c1 sokopay.theagbeko.com        # from anywhere: should show 137.184.192.212
```

If they differ, change the A record for `sokopay` on `theagbeko.com` to the droplet's IP. Certificates can't be issued until it matches.

5. Pull the code from GitHub

The repository **SokoPay/sokopay-platform** is currently **public**, so the droplet clones it over HTTPS with no key or password:

```
sudo mkdir -p /opt/sokopay && sudo chown deploy:deploy /opt/sokopay
git clone https://github.com/SokoPay/sokopay-platform.git /opt/sokopay
cd /opt/sokopay && git log -1 --oneline
```

The droplet only ever **reads** from GitHub. It can't push, and no GitHub credentials are stored on it.

5.1 If the repository is made private again

`git pull` (and `update.sh`) will then fail with "Repository not found" or ask for a username. Give the droplet a **read-only deploy key** and point the clone at it; nothing else changes.

```
ssh-keygen -t ed25519 -C "droplet-sokopay-deploy" -f ~/.ssh/sokopay_deploy -N ""
cat >> ~/.ssh/config <<'EOF'
Host github-sokopay
    HostName github.com
    User git
    IdentityFile ~/.ssh/sokopay_deploy
    IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
cat ~/.ssh/sokopay_deploy.pub          # copy this line (starts with ssh-ed25519)
```

On GitHub, open the repository, then **Settings**, then **Deploy keys**, then **Add deploy key**: - Title: `DigitalOcean droplet (pilot)`. Paste the key. - **Leave "Allow write access" unticked**, then click **Add key**.

Then switch the existing clone over to the key:

```
ssh -T git@github-sokopay          # first time: type "yes"
git -C /opt/sokopay remote set-url origin git@github-sokopay:SokoPay/sokopay-platform.git
git -C /opt/sokopay pull --ff-only
```

The separate `github-sokopay` alias means the key is used only for SokoPay; the other projects' keys are unaffected.

`/opt/sokopay` now holds the full repository. Everything SokoPay needs on the server lives in this folder.

6. The `.env` file (settings and secrets)

All configuration lives in **one file**: `/opt/sokopay/deploy/.env`. Docker Compose and the containers read it. - Git ignores it, so it's never pulled or pushed. - Its permissions are `600`. - Never email it or paste it into chat.

6.1 Create it and generate the secrets

```
cd /opt/sokopay/deploy
bash scripts/make-secrets.sh
```

This copies `.env.example` to `.env` and fills every `CHANGE_ME` with a strong random value. The generated secrets:

Variable	What it is
<code>SECRET_KEY</code>	Django signing key (sessions, tokens, signed links)
<code>FIELD_ENCRYPTION_KEY</code>	Encrypts Ghana Card numbers. If you lose it, that data can't be read.
<code>POSTGRES_PASSWORD</code>	SokoPay's own database password (also in <code>DATABASE_URL</code>)
<code>REDIS_PASSWORD</code>	SokoPay's Redis password (also in <code>REDIS_URL</code>)
<code>USSD_SHARED_SECRET</code>	The USSD gateway must send this (section 12.4)
<code>RAIL_WEBHOOK_SECRET</code>	Signs the mock partner's callbacks

6.2 Check the domain, port and profile

The template already has these values for `sokopay.theagbeko.com`. Open the file, set `ACME_EMAIL` to your email, and confirm the rest:

```
nano .env
```

```
SOKOPAY_DOMAIN=sokopay.theagbeko.com
ACME_EMAIL=you@yourcompany.com
ALLOWED_HOSTS=sokopay.theagbeko.com,localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=https://sokopay.theagbeko.com
QR_BASE_URL=https://sokopay.theagbeko.com
SOKOPAY_WEB_PORT=8100
# Shared droplet (Nginx/Caddy already on 80/443): localdb only
COMPOSE_PROFILES=localdb
```

Save with `Ctrl+O`, then exit with `Ctrl+X`. Check that nothing is left unfilled; the following should print nothing:

```
grep CHANGE_ME .env
```

6.3 What the other settings mean

Variable	Pilot value	Notes
DJANGO_SETTINGS_MODULE	config.settings.staging	Production hardening plus the mock partner. Real production uses <code>config.settings.prod</code> (section 14).
COMPOSE_PROFILES	localdb	Add <code>,caddy</code> only when nothing else uses ports 80/443 (section 7D).
SOKOPAY_ACTIVE_LICENCE	DEMI	Every feature on, for a full test.
RAIL_PROVIDER , KYC_IDENTITY_PROVIDER	mock	No real money or real identity checks.
SMS_PROVIDER	console or arkese1	<code>console</code> writes codes to the log; <code>arkese1</code> sends real SMS (section 12.2).
ARKESEL_API_KEY , ARKESEL_SENDER_ID	your key, MySokoApp	Needed for <code>arkese1</code> . The server refuses to start with <code>arkese1</code> and no key.
OTP_TTL_SECONDS	600	How long an SMS code stays valid (10 minutes).
PORTAL_2FA_SMS	True	Web portal 2FA texts a code to the user's phone. Authenticator apps and backup codes still work.
PUSH_PROVIDER	console	Set to <code>fcm</code> once Firebase is configured.

Optional: DigitalOcean Managed PostgreSQL instead of the container. Set `COMPOSE_PROFILES=` (empty, or just `caddy`) and paste the cluster's connection string, keeping `sslmode=require`:

```
DATABASE_URL=postgres://doadmin:<password>@<host>:25060/defaultdb?sslmode=require
```

7. HTTPS: connect SokoPay to the droplet's web server

Use the one option that matches what you found in section 2.

7A. The droplet runs Nginx (most common)

```
sudo cp /opt/sokopay/deploy/nginx/sokopay.conf /etc/nginx/sites-available/sokopay
# The file is already set for sokopay.theagbeko.com and port 8100; edit it only if you changed either.
sudo ln -s /etc/nginx/sites-available/sokopay /etc/nginx/sites-enabled/sokopay
sudo nginx -t && sudo systemctl reload nginx      # test first; the reload doesn't interrupt the other
sites
sudo apt -y install certbot python3-certbot-nginx # skip if certbot is already installed
sudo certbot --nginx -d sokopay.theagbeko.com --redirect -m you@yourcompany.com --agree-tos -n
```

Certbot adds HTTPS **only to the SokoPay site** and renews it automatically, with the same timer as the other projects' certificates.

7B. The droplet runs Caddy (on the host)

Add this block to `/etc/caddy/Caddyfile`, below the existing sites:

```
sokopay.theagbeko.com {
    encode zstd gzip
    request_body {
        max_size 10MB
    }
    reverse_proxy 127.0.0.1:8100
}
```

Then run `sudo caddy validate --config /etc/caddy/Caddyfile && sudo systemctl reload caddy`. Caddy gets the certificate automatically.

7C. The droplet's proxy runs in Docker (Traefik, nginx-proxy, Caddy container)

That proxy can't reach `127.0.0.1` on the host. Give it SokoPay's web container instead: 1. Find the proxy's network: `docker network ls`. 2. Connect SokoPay's web container to it: `docker network connect <proxy-network> sokopay-web-1`. 3. Add a route for `sokopay.theagbeko.com` to `sokopay-web-1:8000`, the way the other two projects are routed (labels for Traefik, `VIRTUAL_HOST` for nginx-proxy).

7D. Nothing else uses ports 80/443

Use SokoPay's own Caddy. Set `COMPOSE_PROFILES=localdb,caddy` in `.env`. Caddy then serves `SOKOPAY_DOMAIN` with an automatic certificate.

8. Build and start SokoPay

```
cd /opt/sokopay/deploy
docker compose build                # ~5 minutes the first time
docker compose up -d db redis
docker compose run --rm web python manage.py migrate --noinput
docker compose up -d
docker compose ps                  # web (healthy), worker, beat, db, redis: "running"
docker ps --format '{{.Names}}'    # the other two projects' containers are still there
```

Check from your computer. All three should load:

```
https://sokopay.theagbeko.com/healthz -> {"status": "ok", ...}
https://sokopay.theagbeko.com/readyz  -> {"status": "ok"}
https://sokopay.theagbeko.com/legal/  -> the legal pages with the orange TEST ENVIRONMENT banner
```

Also open the other two projects' sites, to confirm they still work.

8.1 Load the demo data

```
docker compose run --rm web python manage.py seed_demo > ~/sokopay-demo-credentials.md
chmod 600 ~/sokopay-demo-credentials.md
cat ~/sokopay-demo-credentials.md
```

This creates the same people, shop, agents and activity as `docs/SokoPay-Demo-Guide.pdf`, with one difference. Because this server is on the internet, **every password, PIN and 2FA secret is random**, and the real ones exist only in that file on the droplet. Share them with testers through a password manager, never in chat.

The shop's merchant code is still **GRACEMTN**.

9. Daily operations

Always run these from `/opt/sokopay/deploy` ; they then affect SokoPay only.

Task	Command
Status	<code>docker compose ps</code>
Logs (all SokoPay) / web only	<code>docker compose logs -f --tail 100 / docker compose logs -f web</code>
Restart SokoPay	<code>docker compose restart</code>
Stop / start SokoPay	<code>docker compose stop / docker compose up -d</code>
Django shell	<code>docker compose run --rm web python manage.py shell</code>
SokoPay's memory and CPU	<code>docker stats --no-stream \$(docker compose ps -q)</code>
Back up SokoPay's database	<code>bash scripts/backup.sh</code> (keeps the last 14 in <code>deploy/backups/</code>)
Nightly backups at 02:00	<code>(crontab -l; echo "0 2 * * * cd /opt/sokopay/deploy && bash scripts/backup.sh") \\ crontab -</code>

Health: - The staff portal's **Health** page shows stuck payouts, failed payments, overdue disputes and safeguarding. - DigitalOcean **Monitoring** alerts cover the whole droplet; check them after adding SokoPay.

Restore a backup:

```
docker compose stop web worker beat
docker compose exec -T db dropdb -U sokopay sokopay
docker compose exec -T db createdb -U sokopay sokopay
docker compose exec -T db pg_restore -U sokopay -d sokopay --no-owner < backups/<file>.dump
docker compose up -d
docker compose run --rm web python manage.py db_fingerprint      # ledger sums to zero, no integrity breaks
```

10. Updating to a new version (git pull)

When a new version is pushed to GitHub:

```
cd /opt/sokopay/deploy
bash scripts/update.sh      # pulls main, backs up, rebuilds, migrates, restarts, checks /readyz
BRANCH=some-branch bash scripts/update.sh # to test another branch
```

`update.sh` does `git pull --ff-only` . If someone edited files on the droplet, it stops rather than overwrite them; see `git -C /opt/sokopay status` . Your `.env` is never touched, because git ignores it.

To go back to the previous version:

```
git -C /opt/sokopay log --oneline -5
git -C /opt/sokopay checkout <previous-commit>
docker compose build web && docker compose up -d
```

Restore the backup `update.sh` made only if a migration changed the data.

11. Other things to know on a shared droplet

- **Ports:** only `127.0.0.1:${SOKOPAY_WEB_PORT}` is used on the host, plus 80/443 if you chose 7D.
- **Images:** SokoPay builds `sokopay-backend:local` and uses the public `postgres:16`, `redis:7` and `caddy:2` images. If another project already uses those images, they're shared but not modified.
- **Disk:** `docker system df` shows Docker's usage for all projects. Clean SokoPay's old build layers with `docker builder prune --filter "label=com.docker.compose.project=sokopay"`. **Don't** run a global `docker system prune`.
- **Reboots:** SokoPay restarts by itself after a droplet reboot (`restart: unless-stopped`), like the other projects.

12. Testing with the apps and portals

12.1 Phone apps pointed at the server

On your computer:

```
API_BASE_URL=https://sokopay.theagbeko.com/api/v1 bash mobile/scripts/build_apks.sh
```

The APKs land in `mobile/dist/`. They work on **real phones** because the server has real HTTPS. They're debug-signed and for testing only.

12.2 Sign-in codes (SMS through Arkesel)

The same SMS code system serves the phone apps (sign-in, PIN reset) and the web portals (2FA). Each code is 6 digits, valid for `OTP_TTL_SECONDS` (10 minutes), and works once. A code sent for the app can't be used for the portal, and the reverse.

Before switching on real SMS: the demo people have made-up numbers (`+233200000001` and so on) that may belong to real strangers. Point the accounts you'll test with at your own phones first:

```
docker compose run --rm web python manage.py shell -c "from apps.accounts.models import User;
User.objects.filter(phone='+233200000001').update(phone='+233XXXXXXXXX')"
```

Replace `+233XXXXXXXXX` with your number, then sign in with that number. For the apps, testers can simply sign up with their own numbers.

Switch on Arkesel in `/sokopay/deploy/.env`:

```
SMS_PROVIDER=arkesel
ARKESEL_API_KEY=<your key from the Arkesel dashboard>
ARKESEL_SENDER_ID=MySokoApp
ARKESEL_SANDBOX=False
OTP_TTL_SECONDS=600
PORTAL_2FA_SMS=True
```


Then run `docker compose up -d`. To try the connection first without spending credit, set `ARKESEL_SANDBOX=True`. Arkesel then accepts the messages and shows them in its SMS history report, but doesn't deliver them.

If an SMS doesn't arrive, run `docker compose logs web | grep -i arkesel`. Arkesel's reason (low balance, sender ID not approved, wrong key) is logged there; codes and keys never are.

With `SMS_PROVIDER=console` nothing is sent; read the codes on the droplet:

```
docker compose logs web | grep "SMS:console" | tail -5
```

12.3 Portals

- **Address:** `https://sokopay.theagbeko.com/dashboard/`. Use the logins from `~/sokopay-demo-credentials.md`.
- **2FA:** after the password, the 2FA page texts a 6-digit code to the account's phone. "Send a new code" is allowed every 30 seconds, up to 5 codes an hour. The authenticator-app secret in the credentials file still works too, or print the app's current code: `bash docker compose run --rm web python manage.py demo_2fa_code +233200000004`
- **Mock partner:** `https://sokopay.theagbeko.com/dev/mock-partner/` approves MoMo prompts and payouts. On the server it needs the superuser, signed in with 2FA.

12.4 Adding real merchants, team members and agents

No shell commands are needed; everything is in the portal.

Who	How
A merchant, added by SokoPay	Back office, then Merchants , then + Add merchant (operations or compliance staff). Enter the owner's name and phone, the business details, and upload documents (PDF, JPEG or PNG, up to 5 MB, stored encrypted). The owner gets an SMS with a link to set their own password (valid 3 days). Then Begin review and Approve ; the owner is texted the result and their merchant code.
A merchant, applying on their own	They open <code>https://sokopay.theagbeko.com/dashboard/register/</code> (also linked from the sign-in page): business details, then a code by SMS, a password and optional documents. The application appears in Merchants for review. Set <code>MERCHANT_SELF_SIGNUP=False</code> in <code>.env</code> to hide the page.
A team member (admin, finance, cashier, developer)	The merchant's owner or an admin opens Team in the merchant portal and enters the person's name, phone and role. They get an SMS invite to set their password. Owners and admins can change roles or remove people; removal takes effect immediately.
An agent (back office only)	Back office, then Agents , then Add an agent : name, phone, kiosk name, location. New people get an account created for them; either way they're texted to install the Agent app and sign in with that number (SMS code, then they choose a PIN). They stay pending until you open the agent and click Activate after KYC checks.

Passwords: nobody ever types or sends someone else's password. - **Forgot password:** on the sign-in page, the user gets a reset code by SMS. - **Change password:** on the **Security** page. - **Expired invite:** staff can click **Resend invite** on the merchant's review page, or the owner can on the **Team** page.

Documents are kept in the `media` Docker volume, encrypted with `FIELD_ENCRYPTION_KEY`. `scripts/backup.sh` backs them up next to the database (`*.media.tgz`), and `wipe.sh` deletes them with everything else.

12.5 USSD

```
curl -s -X POST "https://sokopay.theagbeko.com/api/v1/ussd/callback?key=<USSD_SHARED_SECRET from .env>" \
--data-urlencode sessionId=t1 --data-urlencode phoneNumber=+233244000201 --data-urlencode text=
```

12.6 Following the demo guide

Every walkthrough in `docs/SokoPay-Demo-Guide.pdf` works on the server with these changes: - Use `https://sokopay.theagbeko.com` instead of `http://127.0.0.1:8000`. - Use the logins from the credentials file. - Read sign-in codes from `docker compose logs web`.

13. After the test: remove SokoPay (the other projects stay)

- 1. **(Optional) keep evidence:** export what you need from the portal first (audit log CSV, statements, regulatory figures). Download a final backup only if you need it; it contains the test data. `bash scp deploy@<droplet-ip>:/opt/sokopay/deploy/backups/<latest>.dump .`
- 2. **Wipe SokoPay's data:** `bash cd /opt/sokopay/deploy && bash scripts/wipe.sh` Type `WIPE SOKOPAY` to confirm. This deletes: - SokoPay's containers; - its PostgreSQL, Redis and Caddy volumes; - its built image and backups; - `.env`, with every secret; - its backup cron job.

At the end it lists the containers still running; the other two projects should be there. 3. **Remove SokoPay's web server site and certificate.** Use the option you set up in section 7: - **Nginx (7A):** `bash sudo rm /etc/nginx/sites-enabled/sokopay /etc/nginx/sites-available/sokopay sudo nginx -t && sudo systemctl reload nginx sudo certbot delete --cert-name sokopay.theagbeko.com` - **Caddy on the host (7B):** delete the `sokopay.theagbeko.com { ... }` block, then `sudo systemctl reload caddy`. - **Proxy in Docker (7C):** remove the SokoPay route from that proxy's configuration. 4. **Remove the code and keys:** `bash sudo rm -rf /opt/sokopay shred -u ~/sokopay-demo-credentials.md` If you set up a deploy key (section 5.1), also run `rm -f ~/.ssh/sokopay_deploy ~/.ssh/sokopay_deploy.pub`, delete the `Host github-sokopay` block from `~/.ssh/config`, and remove the key on GitHub (repository Settings, then Deploy keys). Consider making the repository **private** again once the test is over (GitHub: Settings, then General, then Danger Zone, then Change visibility). 5. **Remove the DNS record** for `sokopay.theagbeko.com`. If you added swap only for SokoPay and want it gone: `sudo swapoff /swapfile && sudo rm /swapfile`, then delete its line from `/etc/fstab`. 6. **Check the other projects** still work: open their sites and run `docker ps`. 7. **On your computer:** delete any downloaded dumps and credential files. Uninstall the test APKs from testers' phones. Rotate any real credentials you used (Hubtel, Firebase, Sentry).

The droplet itself is **not** destroyed: the other two projects keep running.

14. Moving to real production later

Area	Pilot (this guide)	Production
Settings	<code>config.settings.staging</code> (mock partner allowed)	<code>config.settings.prod</code> : refuses to start with any mock, and needs a configured and certified Korba or Nsano (<code>docs/RAILS-INTEGRATION.md</code>)
Server	Shared droplet (third project)	Dedicated servers, so SokoPay doesn't share a machine with other systems
Data location	DigitalOcean (outside Ghana)	Hosting approved by the Bank of Ghana and the Data Protection Commission
Database	PostgreSQL container	Managed PostgreSQL with failover, point-in-time recovery, and a restore drill
Data	Demo only	Real customers, so the legal documents must be signed off first (<code>docs/legal</code>)

15. Troubleshooting

Symptom	Fix
<code>port is already allocated</code>	Another project uses 8100. Pick a free port for <code>SOKOPAY_WEB_PORT</code> (and in the proxy config), then <code>docker compose up -d</code> .
<code>502 Bad Gateway</code> from Nginx or Caddy	SokoPay's web container isn't up yet, or the port differs between <code>.env</code> and the proxy config: <code>docker compose ps</code> , then <code>curl -s 127.0.0.1:8100/healthz</code> .
Certificate error in the browser	DNS for the subdomain must point at the droplet. Run certbot again (7A), or check <code>journalctl -u caddy</code> (7B).
<code>Bad Request (400)</code>	The subdomain is missing from <code>ALLOWED_HOSTS</code> in <code>.env</code> . Then <code>docker compose up -d</code> .
<code>CSRF verification failed on portal forms</code>	<code>CSRF_TRUSTED_ORIGINS</code> must be <code>https://<subdomain></code> , and the proxy must send <code>X-Forwarded-Proto</code> (the template does).
<code>web</code> keeps restarting	<code>docker compose logs web</code> . Usually a missing <code>.env</code> value or a wrong <code>DATABASE_URL</code> .
The other projects slowed down	<code>docker stats</code> shows who uses what. SokoPay's limits are in <code>deploy/docker-compose.yml</code> (<code>mem_limit</code>). Lower them, or resize the droplet.
<code>git pull</code> refuses (local changes)	Files were edited on the droplet: <code>git -C /opt/sokopay status</code> . Keep your changes elsewhere, then <code>git -C /opt/sokopay checkout -- .</code> .
<code>Repository not found</code> or a username prompt on <code>git pull</code>	The repository was made private again. Set up the deploy key (section 5.1).
<code>Permission denied (publickey)</code> on pull	The deploy key wasn't added on GitHub, or <code>~/.ssh/config</code> lacks the <code>github-sokopay</code> block (section 5.1).
A MoMo payment stays "pending"	Approve it on <code>/dev/mock-partner/</code> as the superuser.